



Code Hardening: Java and Interpreted Languages

CEN429 Secure Programming — Week 5

Asst. Prof. Dr. Uğur CORUH · 16.10.2026

Today

Hour	Section	Topic
1	1–4	Managed languages and the root of injection, SEI CERT · SQL injection Demo 1 · Command injection Demo 2 · Path traversal Demo 3
2	5–7	Unsafe deserialisation Demo 7 · XML/XXE, templates, XSS · Python and JavaScript (ReDoS)
3	8–12	Bytecode Demo 4 · ProGuard/R8 + string obfuscation Demo 5 · Android R8/measurement · SBOM Demo 6 · Project

Learning outcomes (LO.3 / LO.4): recognise and prevent the injection class (SQL, command, path, XML, deserialisation) · obfuscate with ProGuard/R8 and write `-keep` rules · produce an SBOM and manage dependency risk

A managed language largely solves memory bugs; but data mixing with commands (injection) exists in every language.

What We Bring from Earlier Weeks

- **Memory bugs (overflow, use of freed memory, undefined behaviour)** — across the first four weeks we saw how these bugs arise in C/C++ and how they are prevented (**Week 4**)
- **The structure of the SEI CERT rule book** — identifier, non-compliant example, compliant solution and risk (**Week 4**)
- **TOCTOU (time-of-check to time-of-use)** — the gap between checking a file's permissions and actually using it is a race-condition flaw (**Week 2**)

This week: memory bugs → which ones disappear at the **language level** (Section 1); the SEI CERT structure → the **Oracle Java** standard (Section 1); TOCTOU → re-checking the root **after** the file is opened, in path traversal (Section 4).

This Week's Concepts

Each term is defined once, where it first appears in the body; here we only mark **where**.

Concept	Where
Injection	Section 1
The JVM and bytecode	Section 8
Parameterised query	Section 2
Deserialisation	Section 5
XXE	Section 6
Path traversal	Section 4
ProGuard / R8	Section 9
The <code>-keep</code> rule	Section 9
SBOM	Section 12

Background: Managed Languages and Garbage Collection

- **Managed language:** a language that manages memory **automatically** (Java, C#, Python, JS); the programmer never calls `malloc` / `free`.
- **GC (garbage collector):** automatically cleans up objects no longer used, in the background; the programmer never frees memory by hand.
- This single difference → most memory bugs (overflow, use-after-free) **disappear**; Section 1 shows this in detail.

Background: Databases and SQL

- **SQL**: a database query language (`SELECT ... WHERE ...`).
- The application puts user input into the query.
- If put in wrong → **SQL injection** (Section 2).

Background: Reflection and Dependencies

- **Reflection:** finding and calling a class/method **by its name, at run time**
(`Class.forName("...")` , `getMethod("...")`); obfuscation can break this → `-keep` is required (Section 9).
- **Dependency:** an external library your project uses that your own team did not write; even if your own code is secure, a flaw in a dependency affects you (Section 12).

How Do the Demos Work?

- 7 demos, in the `code/week-05` folder — need **Python 3** and **JDK 17+**
- The main part of the SQL demo runs with Python's built-in `sqlite3` module, with nothing to download
- Run: Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`
- For the optional Java parts (JDBC, ProGuard) every demo folder has a `hazir1a` (prepare) script

 **Ethics:** the injection examples run only against the **synthetic** database/files in the demo folder;
never apply these techniques to someone else's system.

1. Managed languages and the root of injection

Short history — managed languages and the gaps that remain

- 1995 — **Java/JVM**: most memory bugs end at the **language level**
- 1998 — SQL injection is documented · **2003 OWASP Top 10**
- 2002 — **ProGuard** (later **R8**): because bytecode is easy to reverse
- 2015 deserialisation · **2020–21** SolarWinds and **Log4Shell** → the **SBOM** era

The language solves **memory bugs**; it does **not solve injection or dependency risk**.

What does a managed language solve?

- Automatic memory management (GC) → overflow, UAF, double free are **largely gone**.
- Array access is bounds-checked → `ArrayIndexOutOfBoundsException` (a crash, not exploitation).
- Type safety is stricter.

What does a managed language NOT solve?

- **Injection** (data = command) — exists in every language.
- **Logic errors**, authorisation errors.
- **Dependency** vulnerabilities.
- **Deserialisation** attacks.

Memory bugs went down; the new front is injection and the supply chain.

Managed language: solves / does not solve — diagram

Yönetilen dil neyi çözer, neyi çözmez?

Java/Kotlin/C# çöp toplayıcı + sınır denetimi

ÇÖZER (dil düzeyinde)

tampon taşması
use-after-free
çift serbest bırakma
ilklendirilmemiş değişken
→ bellek güvenliği

ÇÖZMEZ

enjeksiyon (SQL, komut, XML)
güvensiz deserialization
koda gömülü sırlar
zafiyetli bağımlılıklar
mantık/yetki hataları

Dil bir hata SINIFINI kaldırır; kalan sınıflar için hâlâ siz sorumlusunuz.

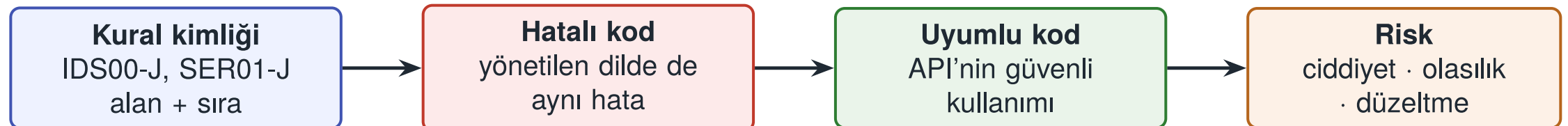
SEI CERT Oracle Java

- A separate **rule book** for Java (CERT Oracle Coding Standard for Java).
- Same structure: noncompliant example + compliant solution + risk.
- Sample areas: injection, serialisation, concurrency, sensitive data.

CERT Oracle Java — diagram

SEI CERT Oracle Java: aynı yapı, farklı dil

yönetilen dil bellek hatasını çözer, mantık hatasını çözmez



IDS (girdi doğrulama), SER (serileştirme), FIO (dosya), MSC (çeşitli) — enjeksiyon konuları IDS'tedir.

SEI CERT · rules touching this week

Prefix	Category	Rule (example)
IDS	Input validation	IDS00-J SQL · IDS07-J <code>Runtime.exec</code> · IDS16/17-J XML
FIO	Input/output	FIO16-J: canonicalise path names
SER	Serialisation	SER12-J: do not deserialise untrusted data
MSC	Miscellaneous	MSC03-J: do not embed sensitive information in code
ERR	Error handling	ERR01-J: do not leak sensitive information in an exception

IDs end with the `-J` suffix.

Sensitive data in Java

- Java `String` is **immutable** → it cannot be erased from memory.
- For a password/key use `char[]`, and **fill** it (`Arrays.fill`) once you are done.
- The GC does **not guarantee** when it will collect.

The common root of injection

Injection = **data** mixing with **command**.

- User data goes to an interpreter (SQL, shell, XML) as a **command**.
- The root is the same; the target (SQL/shell/XML) changes.

The root of injection — data or code? — diagram

Enjeksiyonun ortak kökü: veri ile komutun karışması

bütün enjeksiyon türleri aynı şemanın farklı yorumlayıcılardaki hâli

HATALI: tek kanal

"...WHERE x='" + girdi

Yorumlayıcı girdiyi
KOMUT olarak ayrıştırır

SQLi · komut enj. · XXE
şablon enjeksiyonu

DOĞRU: ayrı kanal

WHERE x = ? (parametre)

Komut SABİT kalır,
girdi yalnız VERİ

enjeksiyon ailesi
tümüyle kapanır

Kaçış (escaping) son çaredir; asıl çözüm komutu ve veriyi ayrı kanala koymaktır.

The common root of the fix

The same principle in every injection type:

- **Separate data from code** (a parameterised API).
- Never embed data into a command via **string concatenation**.
- Allow list + context-appropriate escaping (last resort).

One channel or two channels? — table

Interpreter	One channel (wrong)	Two channels (correct)
SQL engine	<code>"...WHERE name='" + name + "'"</code>	<code>PreparedStatement + ?</code>
Shell	<code>exec("sh -c '...' " + host)</code>	<code>ProcessBuilder(...)</code> — no shell
File system	<code>new File(root, request)</code>	Canonicalise + root check
XML	Building XML by string concatenation	Building elements with a DOM/StAX API

Same principle, four different interpreters.

Injection families (today)

- SQL injection
- Command injection
- Path traversal
- Deserialisation
- XML/XXE, templates, XSS

All share the **same** root and the **same** fix logic.

Rule of this section

A managed language solves the **memory** bug; it does not solve **injection**.

- The root of injection is always the same: data = command.
- The fix is always the same: **separate** data from code.
- We will now see this principle applied to each injection type in turn.

Section 1 — quick check

1. Which error class does a managed language largely solve?
2. What does it not solve?
3. What is the common root of injection?

Section 1 — answers

1. **Memory-safety** bugs (buffer overflow, use-after-free, double/dangling pointer) — through the GC + bounds checking.
2. **Logic/injection/authorisation** bugs, unsafe deserialisation, weak cryptography, **dependency** flaws — these are above the language, unsolved.
3. Untrusted **DATA** mixing with **CODE/command** in the same channel; the interpreter parses the data as a **command**.



2. SQL injection

Buggy code

```
String q = "SELECT * FROM users WHERE name = '" + name + "'";  
stmt.executeQuery(q);    // WRONG: name leaks into the query
```

What if `name` contains a quote and SQL?

Attack · example

The user enters this as `name` :

```
' OR '1'='1
```

The query becomes:

```
SELECT * FROM users WHERE name = '' OR '1'='1'
```

`'1'='1'` is always true → **all rows** are returned.

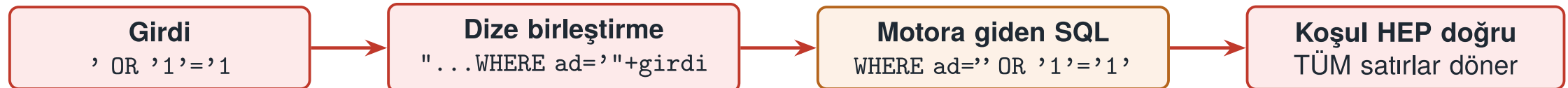
Why did it happen?

- `name` was concatenated into the query **text**.
- The quote closed the data and moved into the **command** part.
- Data and command got **mixed**.

SQL injection step by step — diagram

SQL enjeksiyonu adım adım

girdi tırnağı kapatıp kendi mantığını ekliyor



Parametrelili sorguda motor komutu ÖNCE derler; girdi artık mantık değiştiremez.

Correct · parameterised query

```
PreparedStatement ps =  
    con.prepareStatement("SELECT * FROM users WHERE name = ?");  
ps.setString(1, name);    // name is only DATA  
ps.executeQuery();
```

? is a **parameter**; `name` is never interpreted as a command.

SQL injection — animation

SQL text so far = `...password = '` `' OR '1' = '1'`

Result: the attacker changed the TEXT. The injected fragment: ' OR '1'='1. The database engine now runs this as SQL CODE.

SQL injection — edge case: even a single quote is harmless in a parameterized query

template is **FIXED** — it looks at no input

SQL text so far =

... WHERE password = ?

parameter value =

'

sent to the engine **SEPARATELY**

printResult prints LOGIN REJECTED — even though the parameter is different text, the query's structure was never broken.

Why does it work?

- The query skeleton is sent (compiled) to the database **beforehand**.
- `ad` is bound afterwards as **data**.
- Even if it contains a quote, it stays data.

Other layers

- **Least privilege:** the application's DB user should do only what is needed.
- **Input validation:** an allow list, regardless.
- If you use an **ORM**, use its parameterised API too (not string concatenation).

Escaping is the last resort

- Manually escaping quotes is **error-prone**; every DB is different.
- The **always**-first priority is a parameterised query.
- Escaping only where parameters are not possible, and carefully.

SQL injection · rule of this section

- **Never** build a query with string concatenation.
- **Always** use a parameterised query (`PreparedStatement` , `?`).
- Use an **allow list** for identifiers such as column/table names.
- Escaping only as a last resort.



3. Command injection

Shell or no shell — diagram

Komut çalıştırma: kabuk var mı yok mu?

komut enjeksiyonunun tek kaynağı: araya giren kabuk

Kabuklu (tehlikeli)

```
Runtime.exec("sh -c ...")
```

kabuk ; | && > yorumlar
girdi KOMUT olur

ProcessBuilder (güvenli)

```
new ProcessBuilder(  
    "prog", arg1, arg2)
```

argümanlar AYRI geçer
kabuk hiç çalışmaz

Yine de argüman enjeksiyonuna dikkat: -- ile seçenek/değer sınırını işaretleyin.

Buggy code

```
Runtime.getRuntime().exec("ping " + host); // HATALI
```

What if `host` contains a shell metacharacter?

Attack · example

As `host` :

```
example.com; rm -rf data
```

The shell runs this as **two commands**: `ping` and `rm` .

(The example is synthetic; never run it.)

Why did it happen?

- The command was handed to a **shell** as text.
- `;`, `|`, `&&` are **command separators** in the shell.
- Data got mixed into the command line.

Correct · argument array

```
new ProcessBuilder("ping", "-c", "1", host).start();
```

- **No shell;** `host` is a single **argument**.
- Metacharacters are not interpreted.

Command injection — animation

Text the shell sees =

java ... Printer Hello

Alice

;

echo

LEAKED

Result: the shell ran TWO commands. The second (injected) command: echo LEAKED.

Command injection — edge case: the same attack hits the allow-list first

``name` passed to isAllowed =`

`Alice ; echo LEAKED`

REJECTED

Result: the function returns right here — `buildSecureArgs/ProcessBuilder` is NEVER called; the argv list's immunity is not even tested for this input.

Additional rules

- Where possible, do **not** run an external command at all; use a library API.
- If you must: a fixed path, an argument array, an allow list.
- Similar to ENV33-C : calling `system()` /a shell.

Command injection · rule of this section

- Where possible, **never** run an external command.
- If you must: a shell-less **argument array** (`ProcessBuilder`).
- A fixed path + allow list; watch input starting with `-` (argument injection).



4. Path traversal

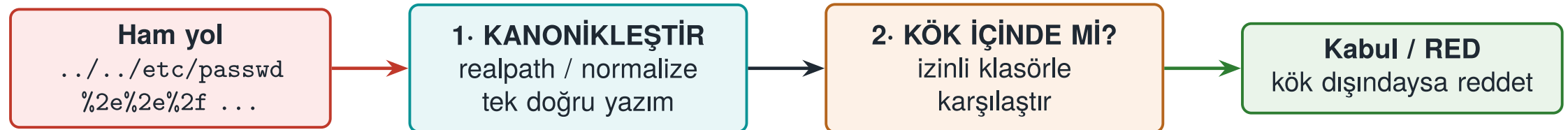
Path traversal

- **Path traversal:** escaping **outside** the allowed folder with `../`.
- Like `files/../../etc/passwd`.
- Prevented with canonicalisation + a root check.

Two steps that close off path traversal — diagram

Yol geçişini kapatan iki adım

sıra önemli: önce kanonikleştir, SONRA denetle



Ham dize üzerinde yapılan denetim kodlanmış biçimlerle atlatılır.

Buggy code

```
File f = new File("uploads/" + name); // WRONG
```

What if `name` contains `../` ?

Attack · example

As `name` :

```
../../../../etc/passwd
```

Result: `uploads/../../../../etc/passwd` → **outside** the allowed folder.

Why did it happen?

- `name` was appended **directly** into the file path.
- The `../` sequence was read by the interpreter (the file system) as "go up a folder."
- There was no check: whether the folder was left was never even asked.

Correct · canonicalise + root check

```
Path root = Paths.get("uploads").toRealPath();  
Path target = root.resolve(name).normalize().toRealPath();  
if (!target.startsWith(root)) throw new SecurityException();
```

Is the canonical path **inside the root**? If not, reject.

Path traversal — animation

request path = 

canonical location relative to root = 

Result: since there is no check at all, the file is read ANYWAY — a file outside the root is leaked.

Path traversal — edge case: the secure version rejects the same attack

request path = 

canonical location relative to root = 

Result: since the canonicalized path lands outside the root, `resolveSafely` returns null — the request is **REJECTED**.

Why does it work?

- First the path is turned **canonical** (single, definite).
- Then we check whether the canonical path is **inside** the root.
- `../`, an encoded form (`../%2f`), or a symbolic link — all of them land on the same real path **after** canonicalisation; none of them can escape the check.

Path traversal · extra

- Restrict the file name to an **allow list** (`[a-zA-Z0-9_.-]`).
- Symbolic links (`symlink`) are resolved by canonicalisation.
- Do not make a user name **directly** into a file name.

Path traversal · rule of this section

- **Never** trust the raw string (`contains("..")` is not enough).
- First **canonicalise**, then check whether it is **inside the root**.
- Resolve symbolic links too (`toRealPath()`); repeat the check after the file is opened as well.

Three injections · common lesson

Type	Wrong	Correct
SQL	String concatenation	Parameterised query
Command	Text to the shell	Argument array
Path	Concatenating <code>../</code>	Canonicalise + root check

Same root: **separate data from code/command**.

Sections 2–4 — quick check

1. Why does `' 0R '1'='1` return all rows?
2. Why is a `ProcessBuilder` argument array safe?
3. Which two steps close off path traversal?

Sections 2–4 — answers

1. Once joined into the string it makes the `WHERE` condition **always true** → all rows (data was parsed as code).
2. Arguments pass **to the program, not the shell**, one by one; there is no metacharacter interpretation (`;` `|`) → command injection is closed off.
3. (1) Convert to a **canonical path** (`realpath /normalise`), (2) verify it is inside the allowed **root directory** (an allow list). Never put user input directly into a path.

5. Unsafe deserialisation

Serialisation

- **Serialisation:** converting an object to a byte sequence (to save or send it).
- **Deserialisation:** converting the bytes back into an object.
- Deserialising untrusted bytes is **dangerous** (code execution).

Buggy code

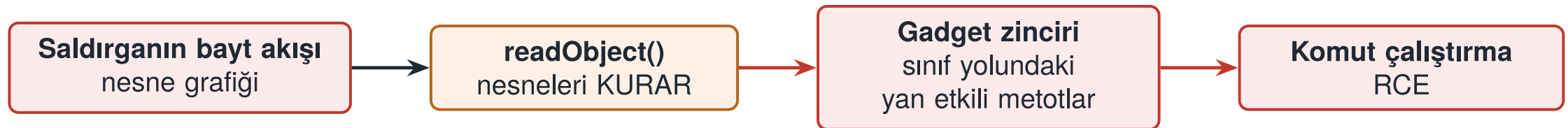
```
Object o = new ObjectInputStream(giris).readObject(); // TEHLİKELİ
```

What if `giris` comes from an untrusted source?

Deserialisation gadget chain — diagram

Güvensiz seri durumdan çıkarma

saldırgan kod yazmaz — VAR OLAN sınıfları zincirler



Savunma: `ObjectInputFilter` ile izin listesi ve sonda “!*” (varsayılan-reddet).

The problem

- Turning untrusted bytes into an object is **dangerous**.
- In Java, `ObjectInputStream.readObject()` can run **code** while unpacking (gadget chains).
- Result: remote code execution.

Why is this so bad?

- Objects' **constructors/methods** run during unpacking.
- If suitable libraries (gadgets) are present, they can be chained to run commands.
- Input that looks like "just data" behaves like **code**.

Correct · best: do not do it at all

- Do not deserialise untrusted data with **native serialisation**.
- Use a **JSON/data** format + a strict schema instead.
- Read only the fields you expect.

If you must · filter

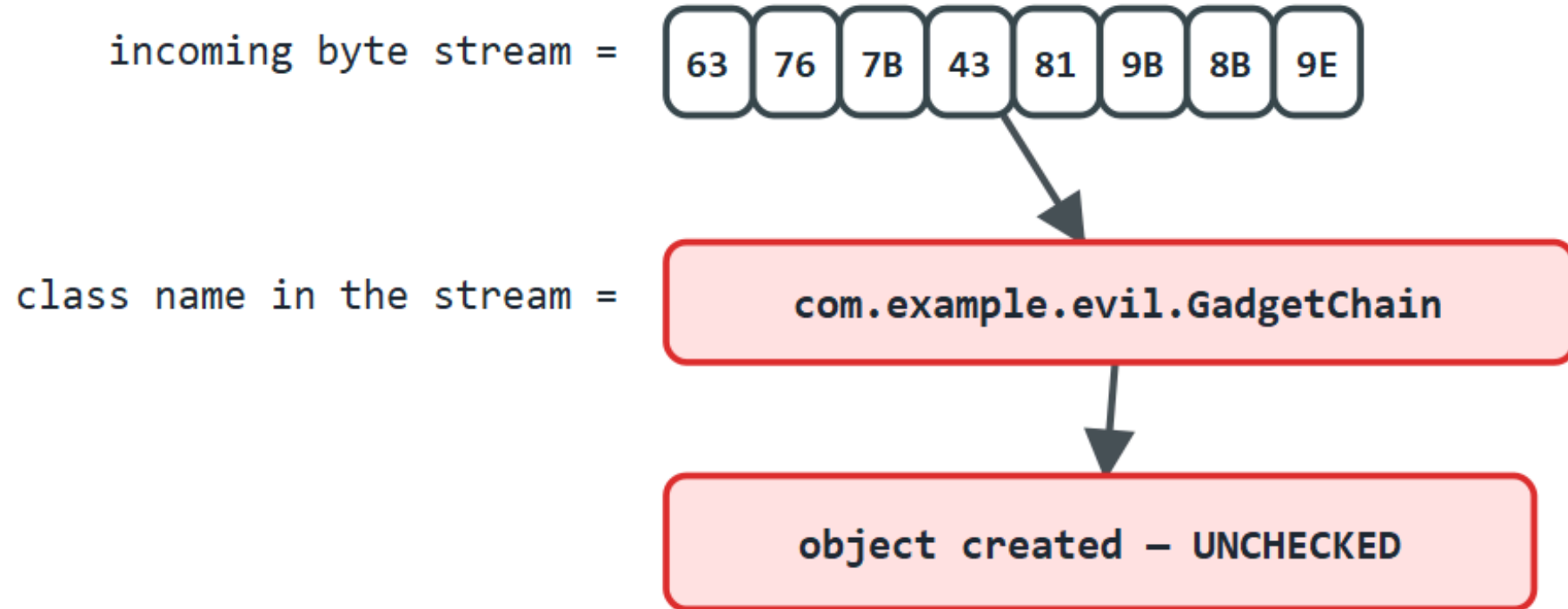
```
ObjectInputFilter f = ObjectInputFilter.Config  
    .createFilter("com.app.*;!*"); // allowed classes only  
ois.setObjectInputFilter(f);
```

If a single class is expected, a narrower pattern also works:

```
ois.setObjectInputFilter(ObjectInputFilter.Config  
    .createFilter("com.app.Session;!*")); // this class only
```

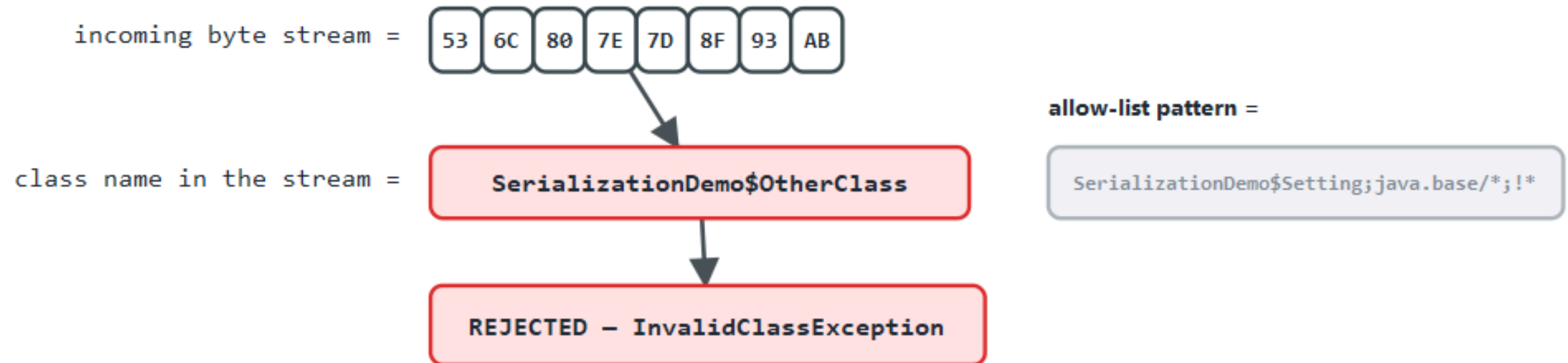
- **Allow list:** only the expected classes.
- Also set a depth/size limit (against DoS).

Deserialization — animation



NO filter: whatever class is in the stream gets instantiated, no matter what it is. In a real attack this could have been a "gadget".

Deserialization — edge case: the filter rejects an unexpected class



`SerializationDemo$OtherClass` matches no part of the pattern: the filter throws `InvalidClassException`, the object is never built at all.

Deserialisation · rule of this section

- Deserialising an untrusted byte stream hands the attacker the power to **choose a class**.
- Best: **do not do it at all** (JSON + a strict schema).
- If you must: an **allow-list filter**, a depth/size limit.



6. XML, templates, XSS

XML and XXE

- **XML**: a structured data format (built from tags).
- **XXE (XML External Entity)**: abuse of XML's "external entity" feature → file reading, SSRF.
- External entities are **disabled** in the XML parser.

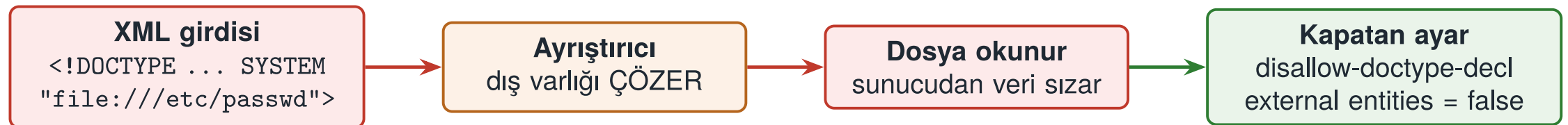
XXE · the problem

- XML lets you define an **external entity**; the parser **resolves** it.
- An attacker can use this to read files or make the server issue a request (SSRF).

XXE — diagram

XXE: XML dış varlık saldırısı

ayrıştırıcının “yardımsever” varsayılını



Kural: XML/YAML/şablon ayrıştırıcılarında gereksiz yetenekleri KAPATIN.

XXE · correct

```
dbf.setFeature(  
    "http://apache.org/xml/features/disallow-doctype-decl", true);  
// ya da dış varlıkları kapat
```

- **Disable external entities**, reject DOCTYPE.
- A one-line setting in most libraries.

Template injection (SSTI)

- Handing user input to a template engine **as the template itself**.
- The engine can run it like code.
- Rule: user input is **data**, the template is **fixed**.

XSS (briefly)

- **XSS**: user data leaking into a web page as **HTML/JS**.
- Fix: **context-appropriate output encoding** (HTML-encode), CSP.
- Same root again: data $\neq$ code.

XML/template/XSS · rule of this section

- Know **which feature of the interpreter is turned on**, and disable what is not needed.
- XML: external entities + DOCTYPE disabled.
- Templates: user data is only a **variable**, never the template itself.
- HTML: encode output **context-appropriately**.

7. The same bugs in Python and JavaScript

The same bugs in interpreted languages — diagram

Yorumlanan dillerde aynı hatalar

Python, JavaScript, Ruby, PHP — kök aynı: veri ile kodun karışması

eval / exec

dizeyi KOD olarak çalıştırır — girdiyle asla birleştirilmez

pickle / unserialize

veriyi nesneye çevirirken kod çağırır — güvenilmez veride kullanılmaz

os.system / shell=True

kabuk ayrıştırır — argüman dizisi kullanın

Düzenli ifade (ReDoS)

geri izleme üstel büyür — desen ve girdi uzunluğu sınırlanır

Dil değişir, kural değişmez: girdiyi asla yorumlayıcıya kod olarak verme.

Same root, different language

- Injection exists in every language: Python `os.system`, JS `child_process`.
- The fix is the same: an argument array, a parameterised query.

Python · examples

- `os.system("cmd " + x)` → command injection; use `subprocess.run([...])`.
- `eval(input)` / `pickle.loads(untrusted)` → **never** with untrusted data.
- SQL: parameterised (`cursor.execute("... ?", (x,))`).

Python · `eval` example (code)

```
value = eval(input_text)           # WRONG: any Python expression

import ast
value = ast.literal_eval(input_text) # Better: literals only
value = int(input_text)             # Best: parse the expected type
```

Rule: user data must **never** reach `eval`.

ReDoS (regular expression DoS)

- A bad **regex** can spend **exponential** time on certain input.
- An attacker can use this to lock up the CPU.
- Fix: a safe regex, an input-length limit, a timeout.

ReDoS · a concrete example

```
Desen:  ^(a+)+$  
Girdi:  "aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!"
```

- The engine tries **millions** of paths to prove there is no match.
- A single request locks the CPU for seconds/minutes (CWE-1333).

JavaScript · prototype pollution

- **Prototype pollution:** an attacker corrupts a property **shared** by all objects, through `__proto__`.
- Unexpected behaviour, authorisation bypass.
- Fix: safe merging, `Object.create(null)`, up-to-date libraries.

Interpreted languages · common rule

- `eval`, `exec`, `pickle`, dynamic `require` → do **not** use with untrusted data.
- Input validation + a parameterised API + up-to-date dependencies.

Sections 5–9 — quick check

1. Why can unsafe deserialisation run code?
2. Which setting disables XXE?
3. What is ReDoS, and how is it mitigated?

Sections 5–9 — answers

1. While an object is being constructed, classes' side-effecting methods (`readObject` / **gadget chain**) fire → an attacker builds a flow using the object graph.
2. The parser setting that disables **external entities and DOCTYPE** (`disallow-doctype-decl` , external entities off).
3. **ReDoS**: a catastrophic-backtracking regex; bad input → **exponential** time → the CPU locks up.
Mitigation: a linear-time engine (RE2), avoid nested quantifiers, a length/time limit.

8. Bytecode and decompilation

The JVM and bytecode

- **JVM (Java Virtual Machine):** the virtual machine that runs Java.
- **Bytecode:** the intermediate form Java source is compiled into (`.class` files).
- Bytecode is machine-independent; the JVM executes it.

Bytecode vs machine code — diagram

Bayt kod neden makine kodundan kolay okunur?

bu yüzden Java'da gizleme (ProGuard/R8) daha kritik

Java bayt kodu

sınıf/metot/alan ADLARI durur
tipler durur
yapı korunur

decompiler $\approx$ kaynağı verir

C/C++ makine kodu

adlar atılır
tipler kaybolur
optimizasyon yapıyı bozar
disassembler $\approx$ ham komut

Gizleme hatayı düzeltmez; yalnız “kolayca okunma” avantajını geri alır.

Java bytecode is easy to read

- `.class` / `.jar` bytecode decompiles back **very close** to the source.
- Tools: `javap` (disassembly), JD-GUI/CFR/Procyon (decompilation).
- Names, strings, structure are largely **visible**.

javap example

```
javap -c -p Application.class    # disassemble the bytecode
```

- Method names, calls, constants are visible.
- This is exactly why obfuscation is **even more** necessary in Java.

javap output — what is visible?

```
private static final java.lang.String VALID_PIN;  
static boolean pinCorrect(java.lang.String);
```

Code:

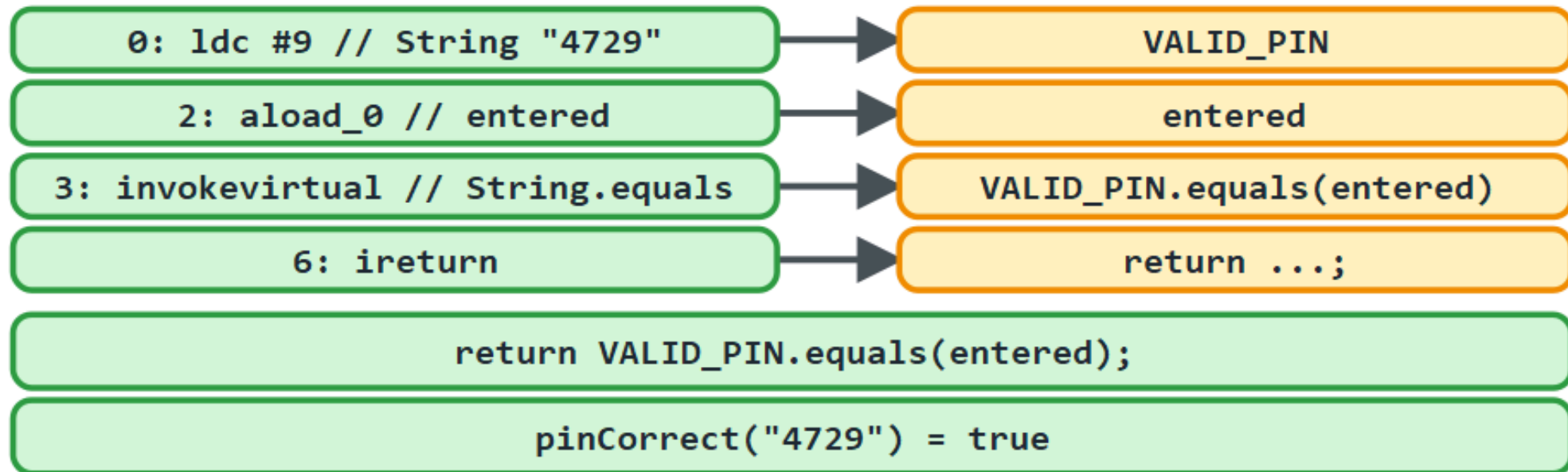
```
0: ldc          #9      // String 4729  
2: aload_0  
3: invokevirtual #11     // String.equals  
6: ireturn
```

There is no source code, but the **constant** (4729), the **names** (VALID_PIN , pinCorrect) and the **logic** ("compare the input against this constant") are plainly visible.

From bytecode to source — animation

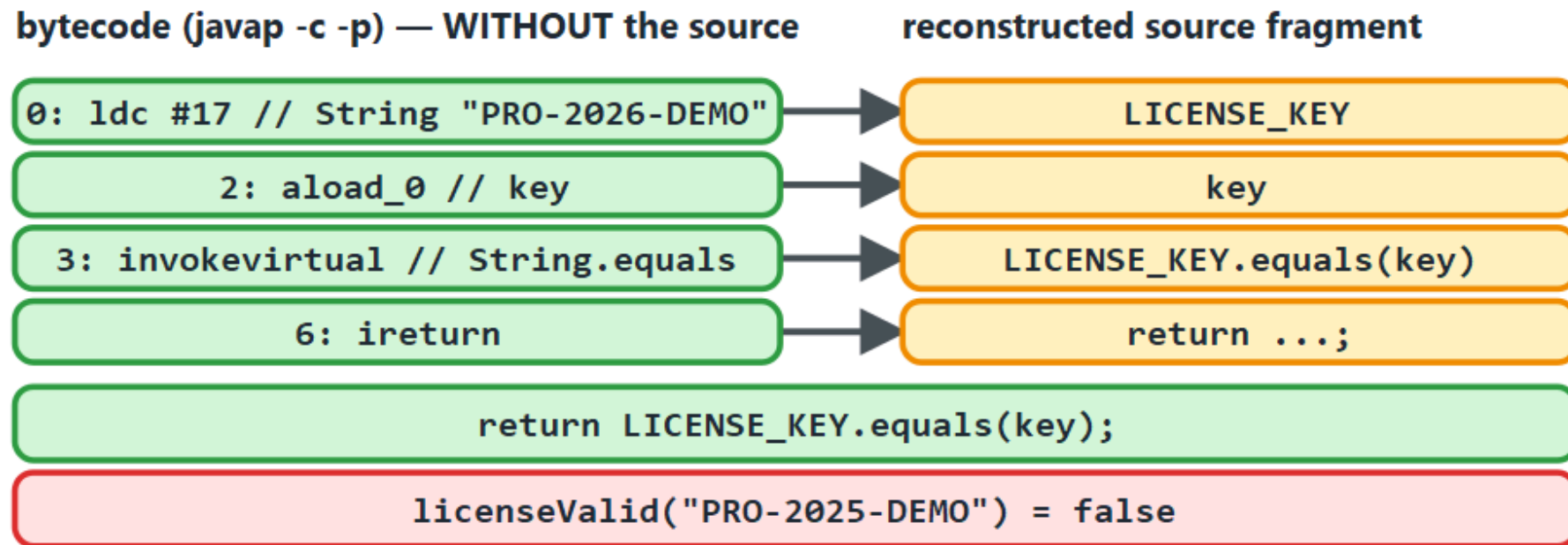
bytecode (javap -c -p) — WITHOUT the source

reconstructed source fragment



Running this logic with "4729": true — the logic we read from the bytecode matches the program's real run-time behaviour exactly.

From bytecode to source — edge case: the license check, wrong year



Running this logic with "PRO-2025-DEMO": false — the logic we read from the bytecode matches the program's real run-time behaviour exactly.

Why is it easier than C?

- Bytecode is **high-level** and carries type information.
- Variable/method names are mostly preserved.
- → reverse engineering is cheap; obfuscation is essential.

There are no secrets on the client

- Obfuscation **makes this harder**, but it does not really **hide** the secret.
- A key/password that does not have to live on the client → keep it **on the server**.
- A secret that **has to** live on the client needs whitebox cryptography (Week 11).



9. ProGuard and R8

R8 and the measurement steps — diagram

Android'de R8 ve gizlemenın etkisini ölçmek

ölçmediğiniz gizleme bir iddiadır



mapping.txt kaybolursa üretimdeki çökme raporlarını bir daha çözemezsiniz.

What do ProGuard/R8 do?

Three jobs:

- **Shrinking:** drops unused code.
- **Optimisation:** simplifies the code.
- **Obfuscation:** makes names meaningless (`a` , `b` , `c`).

ProGuard/R8 · four stages — table

Stage	What it does	Security contribution
Shrinking	Drops unused code	Attack surface shrinks
Optimisation	Inlining, constant folding	Structure moves away from the source
Obfuscation	Turns names into <code>a</code> , <code>b</code> , <code>c</code>	Meaningful names disappear
Preverification	Verification info for older JVMs	—

Name obfuscation example

```
LicenseChecker.verify() → a.b()
```

- Meaningful names disappear.
- The map gets harder for a reverse engineer.
- Removing dead code shrinks the binary.

Why is `-keep` needed?

- Code called **by name** through reflection (an API, entry points, JNI) must be kept.
- If obfuscation changes the name, the call **breaks**.

```
-keep class com.app.Api { public *; }
```

For multiple entry points and annotation-marked members:

```
-keep class com.app.PublicApi { public *; }  
-keepclassmembers class * {  
    @com.app.Reflected *;  
}
```

The **-keep** balance — diagram

-keep dengesi

doğrusu: yalnız dışarıdan erişilen giriş noktalarını keep et

ÇOK keep

reflection, serileştirme,
tüm public API korunur

- adlar değişmez
- gizleme ANLAMSIZ

AZ keep

yalnız dış giriş noktaları

- adlar kısalır
- ama reflection KIRILIR
(çalışma anı hatası)

Her -keep kuralı bir gizleme boşluğudur; en dar kuralı yazın ve test edin.

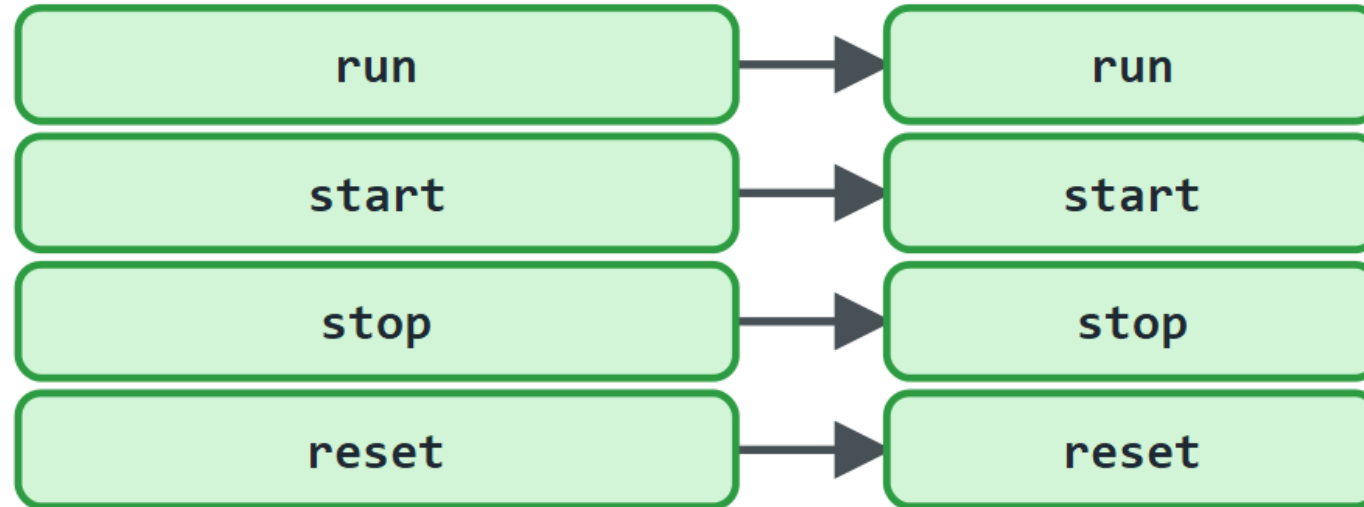
The **-keep** balance

- **Too little -keep** : the application crashes (reflection breaks).
- **Too much -keep** : obfuscation is weakened (everything stays exposed).
- Keep **only the entry points that are genuinely required**.
- **Test** flows that use reflection — any crash after obfuscation?

ProGuard/R8 — animation

-keep rule =

```
-keep class * { run, start, stop, reset; }
```



Result: the -keep rule covers EVERY member — nothing was renamed at all. When a rule is this broad, obfuscation accomplishes nothing.

The mapping file

- R8 produces a `mapping.txt` : obfuscated name → real name.
- It is **kept secret** (not shipped), but it is retained to decode crash traces.
- Archived together with the version identifier.

Mapping file · example (mapping.txt)

```
com.example.payment.CardManager -> com.example.a:  
    android.content.Context context -> a  
    boolean setDefaultCard(java.lang.String) -> b
```

Old name → new name. This file **completely** undoes the obfuscation.

ProGuard/R8 · rule of this section

- Shrink + optimise + obfuscate; it does **not obfuscate strings**.
- `-keep` only for what is **genuinely** called through reflection/JNI.
- `mapping.txt` is a **secret**: keep it, do not distribute it.

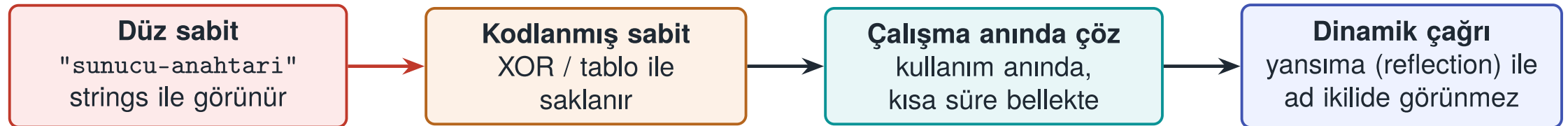


10. String obfuscation and reflection

String obfuscation and dynamic invocation — diagram

Dize gizleme ve dinamik yöntem çağrısı

iki basit yöntem, iki farklı izi siler



Bu bir şifreleme değil, bir GECİKTİRME dir: anahtar da ikilidedir, bulunabilir.

Strings are exposed in Java

- Constant strings inside `.class` appear **in the clear** (`strings` , a decompiler).
- Sensitive URLs, key fragments, messages leak.

String obfuscation

- Strings are **encoded** at build time, decoded at use.
- Same rule as Week 4: it stops `strings`, it does **not keep a secret**.
- The decoding code is also in the bytecode → limited protection.

String obfuscation — animation

key (XOR) = 0x00

encrypted byte array =

57	45	41	4B	2D	4B	45	59	2D	30
----	----	----	----	----	----	----	----	----	----

decrypted string so far =

W	E	A	K	-	K	E	Y	-	0
---	---	---	---	---	---	---	---	---	---

Result: WEAK-KEY-0 decrypted — but since the key is 0x00, the encrypted bytes were already IDENTICAL to the plain text. XORing with a "0" key hides nothing at all.

Reflection conflicts with obfuscation

- A call by name (`getMethod("verify")`) breaks **after** obfuscation.
- Fix: either `-keep` , or drop reflection and call directly.
- Reducing reflection is good for both security and obfuscation.



11. Android R8 and measurement

R8 on Android

- R8 is the **default** shrinker/obfuscator of the Android build.
- `minifyEnabled true` + `proguard-rules.pro` .
- Produces DEX (Android bytecode).

Turning R8 on for the release build

```
android { buildTypes { release {  
    minifyEnabled true  
    shrinkResources true  
    proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),  
        'proguard-rules.pro'  
}}} }
```

The debug build is **not obfuscated**; verify that the shipped build is the **release** one. A plain Java/Gradle project without Android needs only a simpler form:

```
buildTypes { release {  
    minifyEnabled true  
    proguardFiles(..., 'proguard-rules.pro')  
}}
```

Measuring the effect of obfuscation

- Before/after obfuscation:
 - the ratio of meaningful names
 - the count of plain-text sensitive strings
 - APK size
- Measure it and write it into S9 (the measurement logic from Week 9).

Measurement table — extended

Metric	Expected change
Number of classes/methods	Decreases with shrinking
Meaningful-name ratio	Drops sharply with obfuscation
Plain-text sensitive strings	Goes to zero with string obfuscation
Package size	Decreases with shrinking
Number of log calls	Zero in the release
Time to locate a check	Increases (human trial)

Static analysis for Java

- SpotBugs, Error Prone, SonarQube.
- Checks close to the CERT Java rules.
- On every merge, in CI.

Sections 8–13 — quick check

1. Why is Java bytecode easier to read than C?
2. What is the `-keep` balance?
3. Why does reflection conflict with obfuscation?

Sections 8–13 — answers

1. Bytecode is **high-level**; type and **name metadata** (class/field/method) are preserved → a decompiler produces near-source output. C compiles to machine code, discarding names.
2. **The -keep balance**: too much keep → obfuscation is weak; too little keep → reflection/serialisation breaks. Keep only the **external entry points**.
3. Reflection calls **by string** name; once the obfuscator renames it, the string no longer matches → a run-time error. Those names need to be kept (a gap in the obfuscation).



12. Dependency security and SBOM

The problem · supply chain

- Even if your own code is flawless, a flaw in a **library** you use affects you.
- A modern application contains hundreds of dependencies.
- "Which version do I have?" is a critical question.

Real examples

- Log4Shell (Log4j): a single library flaw affected millions of systems.
- A malicious package (typosquatting): a fake package with a similar name.
- An unmaintained/abandoned dependency.

Log4Shell · a concrete example (December 2021)

- A specific expression logged by Log4j 2 loaded and ran a class from a **remote** address.
- CVE-2021-44228, CVSS **10.0** (the highest score).
- The real crisis was not applying the patch: it was the question of **which system had which version**.

What is an SBOM?

- The software's **bill of materials**: which component, which version, which licence.
- When a flaw is announced, "am I affected?" is answered **quickly**.

SBOM formats

- **CycloneDX** (OWASP) — security-focused, widely used.
- **SPDX** (Linux Foundation) — strong on licensing too.
- Generated automatically by tools (during the build step).

A CycloneDX example (JSON, synthetic)

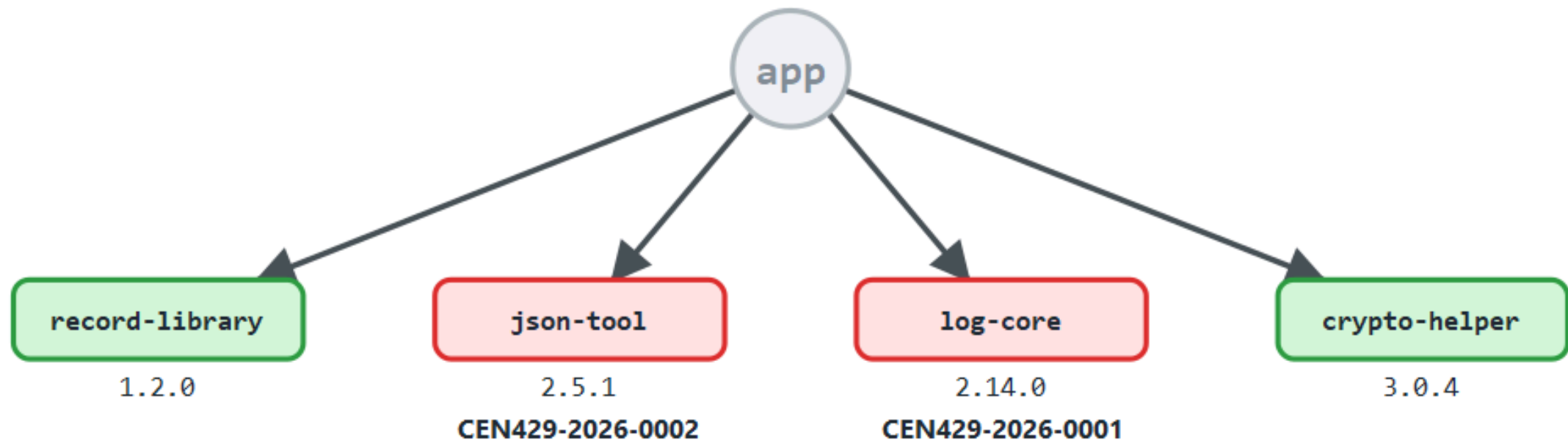
```
{
  "bomFormat": "CycloneDX",
  "components": [{
    "type": "library", "name": "example-json", "version": "1.2.0",
    "purl": "pkg:maven/com.example/example-json@1.2.0"
  }]
}
```

Every component: name, version, **purl** (an ecosystem-independent, uniform identifier), a hash value.

What is VEX?

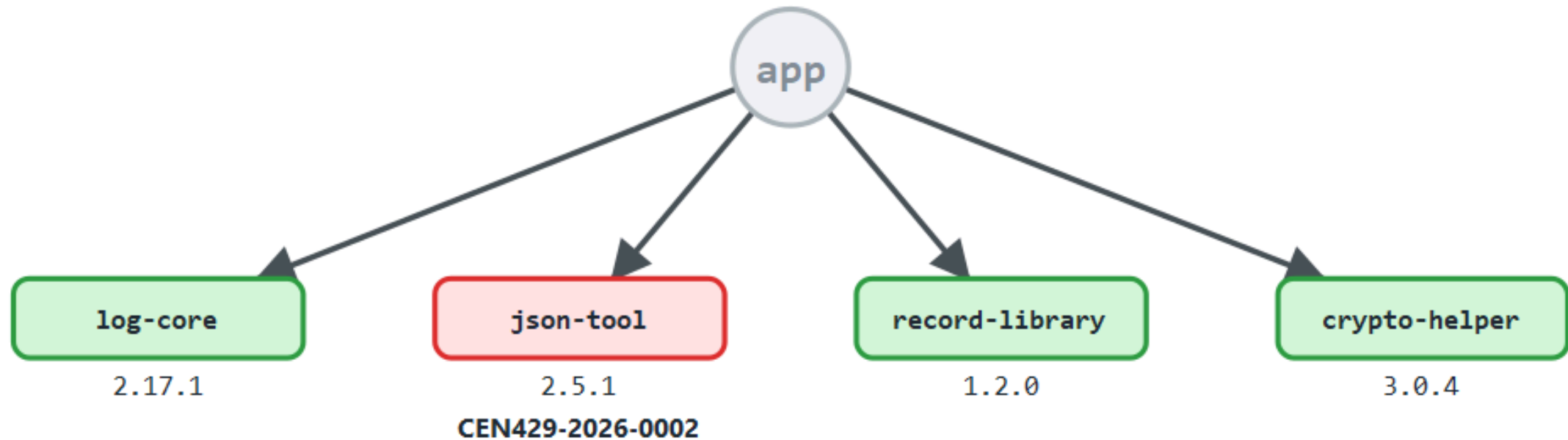
- **VEX (Vulnerability Exploitability eXchange)**: shares "I have this flaw, but it is **not exploitable**" information.
- Reduces noise: not every CVE is a panic.
- Accompanies the SBOM.

SBOM → CVE matching — animation



Result: 2 component(s) were flagged. Without an SBOM, finding this by hand in the code could take days.

SBOM → CVE matching — edge case: the fixed version is no longer flagged



Result: 1 component(s) were flagged. Without an SBOM, finding this by hand in the code could take days.

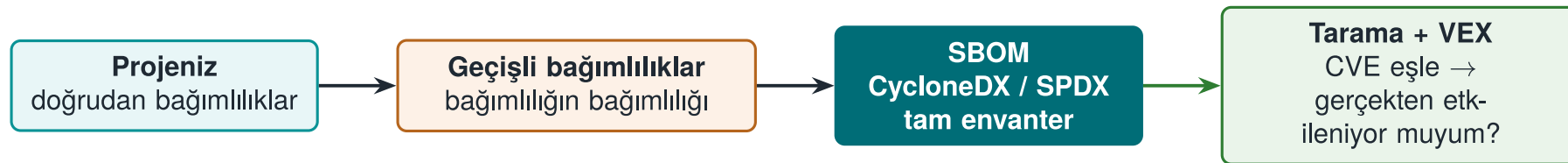
Dependency management rules

- **Pin** versions; do not pull "latest" at random.
- Scan for known vulnerabilities (`dependency-check` , `npm audit` , `pip-audit`).
- A safe update process; apply critical patches fast.

SBOM generation · example flow

Tedarik zinciri: SBOM neden gerekli?

Log4Shell'de asıl sorun “bende var mı?” sorusuydu



Sürümleri sabitleyin: yeniden üretilebilir derleme + beklenmedik güncellemeye karşı koruma.

Each release's SBOM is stored with its release identifier.

```
# örnek: CycloneDX eklentisi
mvn org.cyclonedx:cyclonedx-maven-plugin:makeBom
```

```
dependency-check --scan . --format HTML
```

Link to the project

- **S13:** a secure development process + SBOM.
- An up-to-date SBOM and dependency scan for every release.
- Handed-off requirements: "safe updates" belong to the upstream application (Week 13).

SBOM · rule of this section

- Even if your own code is flawless, **your dependency's flaw affects you**.
- SBOM = the bill of materials; VEX = the answer to "am I genuinely affected?"
- **Pin** the version, **scan** continuously, apply the critical patch fast.

Section 14 — quick check

1. What is an SBOM good for?
2. Why does VEX reduce noise?
3. Why do we pin dependency versions?

Section 14 — answers

1. **SBOM** = a component/dependency inventory; when a new flaw appears it answers "do I have it?" **quickly**.
2. **VEX** states whether the flaw is genuinely **exploitable** in the product; unaffected ones are marked "not affected" → false alarms drop.
3. A pinned version = a **reproducible** build + supply-chain security; it blocks an unexpected/malicious update.

Context · a login form

An application that logs a user in with a username and password.

```
String q = "SELECT * FROM users WHERE name='" + name +  
            "' AND password='" + p + "'";
```

Where is the mistake?

Step 1 · the attacker's input

Into the `name` field:

```
admin' --
```

-- starts a **comment** in SQL; everything after it is ignored.

Step 2 · what does the query become?

```
SELECT * FROM users WHERE name='admin' -- ' AND password='...'
```

The password check stayed **inside the comment** → an `admin` login with no password.

Step 3 · impact

- Authentication was **bypassed**.
- Further: reading other tables with `UNION SELECT`.
- `; DROP TABLE` (if stacked queries are supported) — data loss.

Step 4 · the fix

```
PreparedStatement ps = con.prepareStatement(  
    "SELECT * FROM users WHERE name=? AND password_hash=?");  
ps.setString(1, name);  
ps.setString(2, hash(p));
```

`name` and the password are now **data**; `--` has no effect.

Step 5 · layered defence

- **Never store a password in plain text:** a hash + salt (Week 3).
- **Least privilege:** the DB user only has `SELECT`.
- **Input validation:** an allow list for the username.
- **Logging/monitoring:** failed login attempts.

Lesson

- A single root: data got mixed with the command.
- The real fix: a parameterised query.
- But defence in depth at every layer.

The same bug, three languages

Danger	Java	Python	JS
Command	<code>Runtime.exec(str)</code>	<code>os.system(str)</code>	<code>exec(str)</code>
Safe	<code>ProcessBuilder(...)</code>	<code>subprocess.run(...)</code>	<code>execFile(...)</code>

The same bug, extracting data

Danger	Java	Python	JS
Code execution	<code>readObject</code>	<code>pickle.loads</code> , <code>eval</code>	<code>eval</code> , <code>Function</code>
Safe	JSON + schema	<code>json.loads</code>	<code>JSON.parse</code>

The root is always the same: data $\neq$ code.

Three languages · one rule

- A dynamic "run" (`eval` , `exec` , `pickle` , `readObject`) → **never** with untrusted data.
- Command → an argument array.
- SQL → parameterised.
- Input → an allow list + rejection.

Injection · the general lesson

- Injection is not tied to a language; it is tied to a **pattern**.
- Recognise the pattern: "is user data going to an interpreter as a command?"
- Once recognised, the fix is ready: separate.

Classic Mistakes — Summary

Mistake	Section	Rule
Building SQL/commands with string concatenation	2, 3	Use a parameterised query / an argument array
<code>eval</code> / <code>pickle</code> / <code>readObject</code> with untrusted data	5, 7	Use a data format + schema, never execute it
Leaving external entities open in XML	6	Disable external entities and DOCTYPE
Protecting everything with <code>-keep *</code>	9	Keep only the entry points that are genuinely required
Not scanning dependencies	12	Produce an SBOM, scan continuously

All five were covered earlier in this deck; here they are gathered in one glance.

Project · S9/S13 (the Java side)

- [] Turn injection-prone spots into parameterised APIs (SQL, command, path).
- [] Path: canonicalise + a root check (path traversal closed off).
- [] A filter/schema if there is unsafe deserialisation.
- [] XXE disabled (external entities + DOCTYPE rejected).
- [] Obfuscate with R8/ProGuard; `-keep` only the entry points that are needed.
- [] String obfuscation; no sensitive string in `strings` /a decompiler.
- [] Produce an SBOM + a dependency scan (S13).

Question 1

Which error class does a managed language solve, and which does it not?

Answer: It largely solves memory bugs (overflow, UAF). It **does not** solve injection, logic/authorisation errors, dependency, and deserialisation attacks.

Question 2

Why does `' OR '1'='1'` return all rows?

Answer: The input enters the query through string concatenation; the quote closes the data, and `OR '1'='1'` is always true. A parameterised query prevents this.

Question 3

How does `ProcessBuilder` prevent command injection?

Answer: It supplies arguments as an `array` without using a shell; metacharacters (`;` , `|`) are not interpreted, and the input stays a single argument.

Question 4

Which two steps close off path traversal?

Answer: Canonicalisation (`normalize` / `toRealPath`) + a root check (`startsWith(root)`). Plus a file-name allow list.

Question 5

Why is unsafe deserialisation dangerous? The fix?

Answer: Object methods run while unpacking; code can be run through a gadget chain. Fix: a data format + schema instead of native serialisation; if you must, an `ObjectInputFilter` allow list.

Question 6

What is XXE, and which setting disables it?

Answer: Abuse of XML external entities (file reading/SSRF). The parser setting that disables external entities and DOCTYPE.

Question 7

What are ReDoS and prototype pollution?

Answer: ReDoS: a bad regex spending exponential time (DoS). Prototype pollution: corrupting all objects through `__proto__` in JS. Both need input validation + a safe API.

Question 8

Why does Java bytecode need obfuscation more than most?

Answer: Bytecode is high-level and carries type and name information; it decompiles very close to the source. Names/strings appear in the clear.

Question 9

What is the `-keep` balance?

Answer: Too little `-keep` → reflection breaks, the application crashes. Too much → obfuscation is weakened. Only the entry points genuinely called through reflection are kept.

Question 10

Does string obfuscation keep a key secret in Java?

Answer: No. It stops `strings` /static scanning, but the decoded string is in the clear in memory, and the decoding code is in the bytecode. A key needs different protection.

Question 11

What are SBOM and VEX for?

Answer: An SBOM is the component list; when a flaw is announced it tells you quickly whether you are affected. VEX states whether the flaw is exploitable, reducing noise.

Question 12

What is the common root and fix of every injection type?

Answer: Root: data mixing with command. Fix: **separating** data from code (a parameterised/argument-array API), not building a command with string concatenation.

Glossary

Term	Meaning
Injection	Data = command mixing
Parameterised query	Data is bound separately
Deserialisation	Bytes → object (risky)
XXE	Abuse of an XML external entity
R8/ProGuard	Shrinking + name obfuscation
SBOM/VEX	Component list / exploitability

Next Week

Week 6 — RASP and Run-Time Protection

This week we used obfuscation and shrinking to make bytecode harder to read **statically**; but an attacker can also examine an application **while it runs** (attaching a debugger, dumping memory, installing a hook).

In Week 6 we will see how an application verifies itself at run time with RASP (Runtime Application Self-Protection), and how it detects debuggers and hooks — the next layer built on top of this week's static defences.

This week in one sentence: a managed language solves memory bugs, but **injection exists in every language**; the fix is always separating data from the command, and on the protection side it is obfuscation (R8) and **dependency/SBOM** management.